Time Complexity

Topics To Cover:

1. Basics of Logarithm
2. Iterations and their impact
3. Comparing Algo
4. Time Complexity
5. TLE → Time Limit Exceeded
   ↳ Importance of constraints.

Song: The Other Side
-Bruno Mars

Hi Everyone!!!

Basics of Log

$\log_b a = c$

base

$b^c = a$

* What should be the power of b such that it becomes equal to a. the answer is c

1. $\log_2 8 = 3$
   $2^3 = 8$

2. $\log_2 64 = 6$
   $2^6 = 64$

3. $\log_3 10 = 2.\text{something} \approx 2$ (floor)
   $3^2 = 10$

Q. How many times should we divide 100 by 2 such that we are left with 1.

$$\frac{100}{2} = \frac{50}{2} = \frac{25}{2} = \frac{12}{2} = \frac{6}{2} = \frac{3}{2} = 1$$

Ans: 6

$$\Rightarrow \frac{100}{2^6} = 1 \Rightarrow \boxed{100 = 2^6} \quad \Big| \quad a = b^c$$

$\Rightarrow \log_2 100 = 6$

Q. How many times do I divide 100 by 3 to make it equal to 1.

Ans = $\log_3 100 = 4$

Quiz: N = 9, div by 2 to make it equal to 1

$= \log_2 9 = 3$

Quiz: N = 27 div by 2

$\log_2 27 = 4.\text{something} \approx 4$

Quiz

Q1.
```
i = N
while (i > 1):
    i = i/2
```

Value of i in the while loop

$$\boxed{N \to \frac{N}{2} \to \frac{N}{2} \div 2 = \frac{N}{4} \to \frac{N}{8} \to \frac{N}{16} \to \frac{N}{32} \cdots\cdots\cdots 1}$$

Lets say after K iterations, N becomes 1

$\frac{N}{2^K} = 1$

$= N = 2^K$ | $a = b^c \to \log_b a = c$

$= \log_2 N = K$

No. of iterations

Quiz

for (i goes from 1 to N-1 and gets multiplied with 2):

$i \to 1$ to $N-1$ but $i = i * 2$

i	$i*2$	check $i<N$
1		T
1	2	T
2	4	T
4	8	T
8	16	T
⋮	⋮	⋮
2^{K-1}	2^K	F

K iterations

what should be the value equal to or greater than N to cause loop break ?

$2^K = N$

$= \log_2 N = K$

No. of iter

$b^c = a$

$\Rightarrow \log_b a = c$

Cheat Code :

if a loop iterates such that each time the variable gets multiplied or divided by 2, the no. of iterations will MOSTLY be $= \log_2 N$ times

Quiz : for (i → 0 to N-1 and i * = 2)

i	$i*2$	$<N$
0	0	T
0	0	T
0	0	T
⋮	⋮	⋮

Ans: Infinite loop

Q. for (i → 1 to 10)
  for (j → 1 to N)
  {
  }

10 times

i	j	No of iter
1	[1, N]	N iter
2	[1, N]	N iter
3	[1, N]	N iter
4	⋮	⋮
⋮	⋮	⋮
10	[1, N]	N iter

Ans: 10 N times ⇒ $\Theta(N)$

Quiz

for (i → 1 to N)
  for (j → 1 to N)
  {
  ⋮
  }

Ntimes

i	j	No. of iter
1	[1, N]	N iter
2	⋮	⋮
3	⋮	⋮
⋮	⋮	⋮
N	[1, N]	N iter

Ans: N * N times
= N^2 times
= $\Theta(N^2)$

Quiz

for (i → 1 to N)
  for (j → 1 to N but $j^* = 2$)
  {
  }

$j^* = 2$ every iter

Ntimes

i	j	No. of iter.
1	[1, N]	log N
2	⋮	⋮
3	⋮	⋮
⋮	⋮	⋮
N	[1, N]	$\log_2 N$

Ans: N * $\log_2 N$ ⇒ $\Theta(N \log N)$

Quiz

for (i → 1 to 4)
  for (j → 1 to i)
  {
  }

i	j	No. of iter.
1	[1, 1]	1 iter
2	[1, 2]	2 iter
3	[1, 3]	3 iter
4	[1, 4]	4 iter

$= 1+2+3+4 = 10$

TC is constant

$\Theta(1)$

Gauss Baba

```
for ( i -> 1 to N)
    for ( j -> 1 to i)
    {
    }
```

i	j	No. of ites
1	[1,1]	1
2	[1,2]	2
3	[1,3]	3
⋮	⋮	⋮
N	[1,N]	N

$$= \underbrace{1+2+3+4 \ldots\ldots + N}$$

$$= \frac{N(N+1)}{2} \Big\} \text{ Gauss Baba}$$

$$\frac{N^2}{2} + \frac{N}{2} \hookrightarrow \Theta(N^2)$$

Compare Algo :

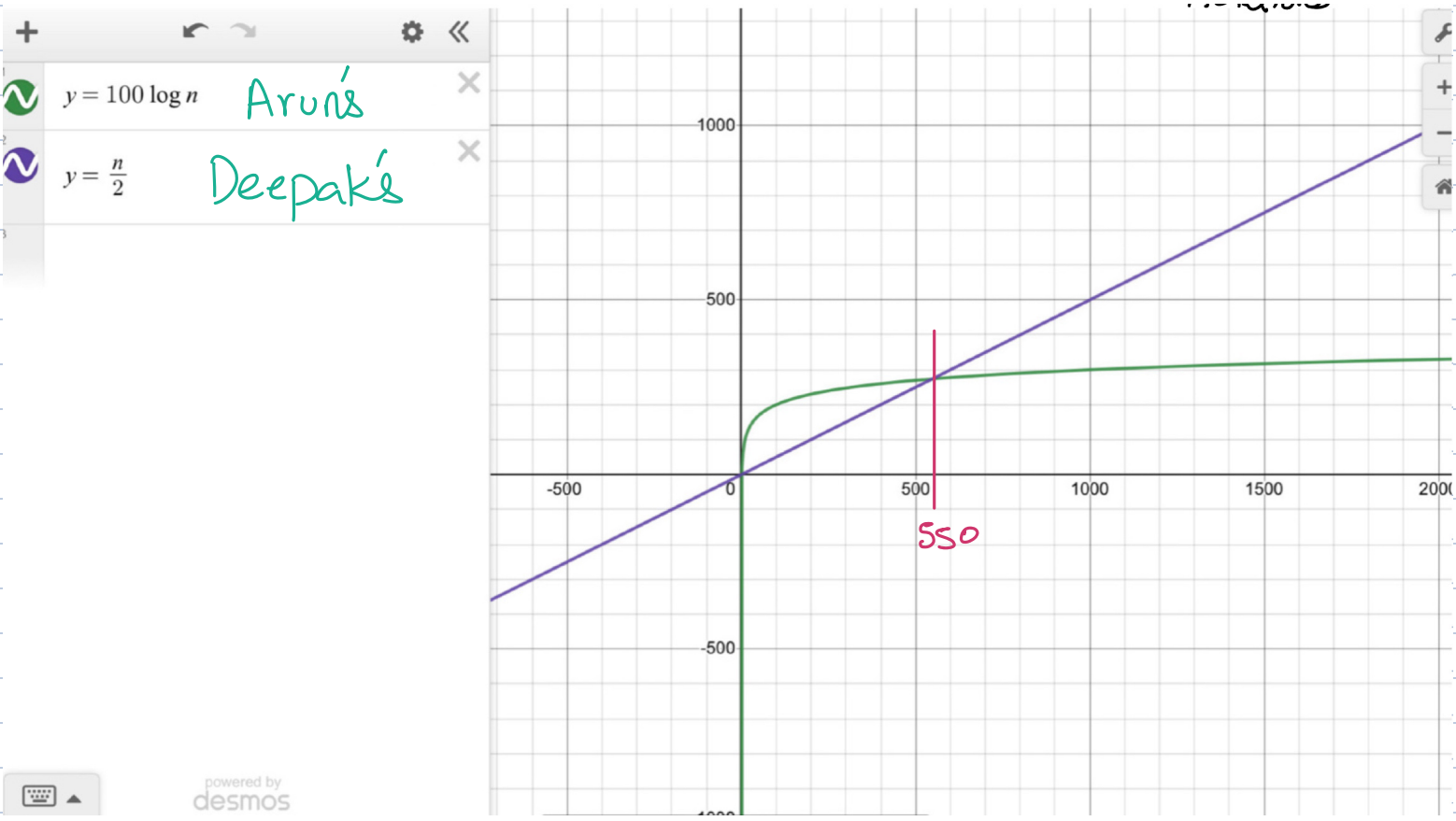
	Arun	Deepak
No. of ites	$100 \log_2 N \Rightarrow \Theta(\log N)$	$\frac{N}{2} \Rightarrow \Theta(N)$
$N = 10$	$100 \log_2 10 = 300$ times	$\frac{10}{2} = 5$ times
$N = 100$	$100 \log_2 100 = 600$ times	$\frac{100}{2} = 50$ times
$N = 10^4$	$100 \log_2 10^4 = 1200$ times	$\frac{10000}{2} = 5000$ times

* After a certain point (≈ 550) Arun's code performs better than Deepak's code.

Song : Baby Shark
↳ 16 B views

* Arun's algo is better as it performs better with large no.

Break: 10:18 - 10:25



Asymptotic Analysis

Big O	Upper Bound	Worst Case
Big Theta Θ	Lower Bound	Best Case
Big Omega Ω	Tight Bound	Avg Case

How To Calculate Big O

1. Count no. of iterations
2. Exclude the lower order term
3. Get rid of constants.

↳ stay HOT → Highest Order Term

Eg: $4N^2 + \underbrace{3N + 500}_{LOT}$

HOT

$4N^2 = \Theta(N^2)$

Quiz: $4N + 3N\log N + 1$

$3N\log N = \Theta(N\log N)$

Comparision for HOT

$\log N < \sqrt{N} < N < N\log N < N\sqrt{N} < N^2 < N^3 < 2^N < N! < N^N$

Quiz: $f(n) = \cancel{4N\log N} + 3N * \sqrt{N} + \cancel{10^6}$

LOT

HOT

$3N\sqrt{N}$

$\Theta(N\sqrt{N})$

Why ignore Lower order Terms?

$f(n) = N^2 + 10N \longrightarrow \Theta(N^2)$

HOT LOT

	N^2	$10N$	N^2+10N	% of LOT $\frac{10N}{\text{Sum}} * 100$
$N=10$	100	100	200	$\frac{100}{200} * 100 = 50\%$
$N = 100$	10000	1000	11000	$\frac{1000}{11000} * 100 = 9\%$
$N = 10^4$	10^8	10^5	$10^8 + 10^5$	$\frac{10^5}{10^8 + 10^5} * 100 = 0.1\%$

* As the value of N goes higher, contribution of LOT become negligible

Why Ignore Constants:

$$\left.\begin{aligned} fn1 &= n \\ fn2 &= 10n \\ fn3 &= 100n \end{aligned}\right\} \Theta(N) \rightarrow \text{linear TC}$$

* The constant generally does not grow substantially of contribute to the final iterations, specially when N is very large.

Big Θ notation has problems

i) Big Θ does not guarantee a better algo for every input.

eg: Deepak's code performs better than Arun's code uptill $N \leq 540$

ii) The constant can be so huge that it outweighs the iterations.

eg: $10^{15} N = \Theta(N)$

This is to impactful to ignore

TLE $\longrightarrow$ Time Limit Exceeded

Online Compilers $\rightarrow$ 1 Ghz $\rightarrow$ 10^8 iters/sec

Expects the code to run under 1sec

Anything beyond 10^8 iters will cause TLE

Constraint $N \leq 10^6$

$\hookrightarrow$ Code a sol. $\longrightarrow$ TC: $\Theta(N^2)$

$(10^6)^2 = 10^{12}$ iterations $\hookrightarrow$ TLE

General lifecycle of an Interview

Interview → Question → Idea → Code → TLE / Pass

Interview → Question → Idea → Code → TLE / Pass

Question → Constraint $N \le 10^3$

Idea → 3 Nested loops → $\Theta(N^3)$ → 10^9 iter → TLE

Idea → 2 Nested loop → $\Theta(N^2)$ → 10^6 iter ✓

find factors :

for $i \to 1$ to N

TC: $\Theta(N)$

for $i \to 1$ to $\frac{N}{2}$

TC: $\Theta(N)$

for $i \to 1$ to $\sqrt{N}$

TC: $\Theta(\sqrt{N})$